Análisis Estructural y Comparativo de Paradigmas en Criptografía Homomórfica: De los Retículos Algebraicos a las Transformaciones de Variable No Lineales

1. Introducción: La Encrucijada de la Computación Privada

La criptografía moderna se enfrenta a un desafío monumental: reconciliar la necesidad imperativa de privacidad de datos con la demanda insaciable de computación intensiva en la nube y el aprendizaje automático. La Encriptación Totalmente Homomórfica (FHE, por sus siglas en inglés) ha sido descrita durante mucho tiempo como el "Santo Grial" de la criptografía, prometiendo la capacidad de realizar cálculos arbitrarios sobre datos cifrados sin necesidad de descifrarlos previamente. Sin embargo, la transición de la teoría a la práctica ha estado marcada por obstáculos significativos relacionados con la eficiencia computacional, la expansión del texto cifrado y, fundamentalmente, la gestión del ruido criptográfico.

Durante la última década, el paradigma dominante ha sido el de los esquemas basados en retículos (*Lattice-based cryptography*), específicamente aquellos fundamentados en la dureza de problemas como el Aprendizaje con Errores en Anillos (Ring-LWE). Esquemas como BGV, BFV y, más recientemente, CKKS (Cheon-Kim-Kim-Song), han establecido el estándar para la computación aritmética segura. No obstante, estos sistemas imponen una estructura algebraica rígida que, si bien preserva homomorfismos de suma y multiplicación, introduce una complejidad asintótica severa cuando se trata de operaciones no lineales o de circuitos de gran profundidad, requiriendo costosos procedimientos de *bootstrapping* para refrescar el presupuesto de ruido.

Este informe presenta una investigación exhaustiva y profunda que contrasta este paradigma establecido con una propuesta emergente y disruptiva: la **Encriptación Homomórfica basada en Transformaciones de Variable**. Basado en la documentación técnica proporcionada [1] y una revisión extensiva de la literatura actual sobre CKKS y aproximaciones polinomiales [2], este análisis desglosa los fundamentos matemáticos, las implicaciones de seguridad y las consecuencias arquitectónicas de este nuevo enfoque. La tesis central que se examinará es si el abandono de las estructuras de anillo tradicionales en favor de transformaciones computables no lineales (E_j) y funciones de evaluación asociadas (P_j^*) puede ofrecer una

vía viable para superar las barreras de eficiencia que actualmente limitan la adopción masiva de FHE.

2. El Paradigma Hegemónico: Retículos, Aritmética Aproximada y el Esquema CKKS

Para valorar adecuadamente la innovación que suponen las transformaciones de variable, resulta indispensable diseccionar primero la anatomía de los esquemas actuales, con un énfasis particular en CKKS, dado que es el estándar actual para el procesamiento de datos reales y aprendizaje automático privado.

2.1 Fundamentos Algebraicos y la Estructura Ring-LWE

Los esquemas de segunda y tercera generación, como CKKS, operan sobre anillos de polinomios ciclotómicos. El espacio de texto plano y el espacio de texto cifrado se definen típicamente sobre el anillo cociente $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$, donde N es una potencia de dos y q es un módulo entero grande.[5] La seguridad de estos sistemas se reduce a la dificultad de distinguir polinomios con ruido aleatorio pequeño de polinomios uniformemente aleatorios en el anillo, una variante del problema *Shortest Vector Problem* (SVP) en retículos ideales.

En el esquema CKKS, la innovación crítica radica en su capacidad para manejar aritmética aproximada. A diferencia de BFV o BGV, que operan sobre enteros exactos, CKKS trata el ruido de encriptación como parte del error de cuantización natural de los números reales.[7]

2.1.1 Codificación SIMD y la Incrustación Canónica

Una de las características más potentes de los esquemas basados en Ring-LWE es la capacidad de empaquetado SIMD (*Single Instruction, Multiple Data*). Un solo polinomio cifrado puede contener un vector de $N/2$ números complejos. Esto se logra mediante la **Incrustación Canónica** (*Canonical Embedding*).

El proceso de codificación en CKKS transforma un vector de mensajes $\mathbf{z} \in \mathbb{C}^{N/2}$ en un polinomio $m(X) \in \mathcal{R}$.[8] Matemáticamente, esto implica:

1. Aplicar la transformada inversa discreta de Fourier (iDFT) o una variante negacíclica a $\mathbf{z}$ para obtener coeficientes polinomiales.
2. Escalar estos coeficientes por un factor Δ (usualmente 2^{30} a 2^{60}) para preservar la precisión decimal al redondear a enteros.

3. Redondear al entero más cercano para obtener $m(X) \in \mathbb{Z}[X]$.

La relación fundamental es que la evaluación del polinomio en las raíces de la unidad correspondientes recupera los valores del vector original (salvo el error de redondeo).[10]

2.2 La Dinámica del Ruido y el Costo de la Multiplicación

El talón de Aquiles de todos los esquemas basados en LWE es el ruido. Cada texto cifrado c contiene un término de error e. Las operaciones homomórficas afectan este error de maneras distintas:

- **Adición:** El ruido se suma linealmente ($e_{add} = e_1 + e_2$). Esto es computacionalmente barato y tiene un impacto mínimo en el "presupuesto" de ruido.[12]
- **Multiplicación:** El ruido crece de forma superlineal. Al multiplicar dos textos cifrados $c_1 \cdot c_2$, el término de error resultante contiene productos cruzados del mensaje y el error ($m_1 e_2 + m_2 e_1 + e_1 e_2$). Esto no solo aumenta la magnitud del error, sino que también eleva el factor de escala a Δ^2.[13]

2.2.1 Reescalado (*Rescaling*) y Profundidad Multiplicativa

Para manejar la explosión del factor de escala, CKKS introduce la operación de *Rescaling*. Esta operación divide el texto cifrado por un entero cercano a Δ, reduciendo tanto el mensaje como el ruido, y consumiendo un nivel del módulo q.[13] Si se dispone de una cadena de módulos $q_L > q_{L-1} > \cdots > q_0$, cada multiplicación seguida de un reescalado reduce el nivel del texto cifrado de l a $l - 1$.

La **profundidad multiplicativa** (L) es, por tanto, un parámetro fijo determinado en la generación de claves. Una vez que un texto cifrado alcanza el nivel 0, no se pueden realizar más multiplicaciones sin corromper el mensaje, a menos que se aplique *bootstrapping*.[12]

2.3 El Cuello de Botella del Bootstrapping en Retículos

El *bootstrapping* es el proceso de "refrescar" un texto cifrado agotado, permitiendo una profundidad de circuito teóricamente infinita. En CKKS, esto implica evaluar homomórficamente el circuito de descifrado. Sin embargo, debido a la estructura aproximada, esto es notablemente más complejo que en esquemas exactos.

El procedimiento estándar de *bootstrapping* en CKKS consta de pasos intensivos [15]:

1. **ModRaise:** Elevar el módulo del texto cifrado (introduciendo un error grande que debe ser eliminado).
2. **CoeffToSlot (CTS):** Una transformación lineal homomórfica que mueve los coeficientes del polinomio a los "slots" de mensaje para poder operarlos. Esto requiere multiplicaciones de matrices costosas y rotaciones.[17]
3. **EvalMod:** La evaluación homomórfica de la función modular ($x \pmod q$) para eliminar el error introducido en ModRaise. Dado que la función modular no es polinomial, se debe aproximar utilizando polinomios de muy alto grado (a menudo mediante series de Taylor o Chebyshev de funciones trigonométricas como $\sin$).[18]
4. **SlotToCoeff (STC):** La transformación inversa a CTS.

Este proceso consume una cantidad significativa de la capacidad del circuito (profundidad) y es la operación más lenta en la computación FHE, a menudo tomando segundos o minutos por operación dependiendo de la precisión requerida.[16]

2.4 El Desafío de las Funciones No Lineales (Exp, Softmax, ReLU)

Mientras que las operaciones lineales son eficientes en CKKS, las funciones no lineales requeridas por el aprendizaje profundo (como ReLU, Sigmoide, y Softmax) representan un obstáculo formidable. La encriptación homomórfica basada en retículos solo soporta nativamente la suma y la multiplicación. Por lo tanto, cualquier función no lineal $f(x)$ debe ser sustituida por una aproximación polinomial $P(x)$.[3]

2.4.1 Aproximación Polinomial y Trade-offs

Para aproximar funciones como e^x (necesaria para Softmax) o $\mathrm{ReLU}(x)$, se utilizan series de Taylor o aproximaciones de Chebyshev.[4]

- **Precisión vs. Profundidad:** Para lograr una alta precisión en un intervalo amplio, se requiere un polinomio de alto grado. Sin embargo, evaluar un polinomio de grado d consume $\lceil \log_2 d \rceil$ niveles de profundidad multiplicativa.[3]
- **Estabilidad Numérica:** La evaluación de e^x en dominios cifrados es propensa a desbordamientos (*overflow*) y pérdida de precisión catastrófica debido a la acumulación de ruido CKKS.[21] Algoritmos avanzados utilizan estrategias de "normalizar y elevar al cuadrado" para mitigar esto, pero el costo computacional sigue siendo $O(\log n)$ en profundidad multiplicativa.[3]

El análisis de la literatura confirma que la evaluación de una capa Softmax en un modelo de lenguaje grande (LLM) bajo CKKS puede tomar desde segundos hasta minutos, dependiendo de la optimización y el hardware, lo que limita severamente la aplicabilidad en tiempo real.[22]

3. Propuesta Disruptiva: FHE basado en Transformaciones de Variable

En contraste con la aproximación de fuerza bruta de los retículos (usar polinomios de grado 100+ para simular funciones no lineales), el esquema propuesto en el documento "Paper_FHE_2_0" [1] plantea un cambio de paradigma: **modificar la estructura matemática del cifrado para que la operación no lineal sea nativa o de bajo costo.**

3.1 Definición Formal del Esquema Π

El esquema se define por la tupla $\Pi = (\mathrm{KeyGen}, \mathrm{Enc}, \mathrm{Eval}, \mathrm{Dec})$, pero su núcleo reside en la naturaleza de las funciones de cifrado y descifrado. En lugar de un homomorfismo de anillo $\phi : \mathcal{R} \to \mathcal{R}'$, el esquema utiliza un conjunto de **transformaciones computables no lineales** $\{E_j\}_{j=1}^k$ y funciones de evaluación asociadas $\{P_j^*\}_{j=1}^k$.[1]

La propiedad fundamental que rige el esquema es:

$$P(x) = P_j^*(E_j(x))$$

Donde x es el dato en claro, E_j es la transformación de cifrado seleccionada, y P_j^* es la función de reconstrucción que reside en la clave secreta. Esto habilita la **desencriptación funcional**: el descifrador obtiene $P(x)$ sin necesariamente revelar x, una propiedad de privacidad (f-privacidad) que es difícil de garantizar en los esquemas FHE tradicionales donde el descifrado devuelve el mensaje plano completo.[1]

3.2 Instanciaciones Concretas: Logaritmos y Polinomios

El documento detalla cómo instanciar este marco abstracto para resolver los problemas específicos de la multiplicación y la no linealidad que plagan a CKKS.

3.2.1 La Solución Logarítmica a la Profundidad Multiplicativa

Para la multiplicación de valores positivos ($x, y > 0$), el esquema propone una

transformación logarítmica $\Sigma_j(x)$ [1]:

$$\Sigma_j(x) = (\log(x + \delta_1), \log((x + \delta_2)^2), \ldots, \log((x + \delta_k)^k), r_j)$$

Donde $\delta_i > 0$ son desplazamientos de seguridad y r_j es aleatoriedad.

La operación homomórfica para la multiplicación ($x \cdot y$) se transforma en una **suma vectorial** en el espacio cifrado:

$$(E_j(x) \otimes E_j(y))_i = E_j(x)_i + E_j(y)_i$$

Análisis Crítico: Esta es una inversión fundamental de la jerarquía de costos. En los retículos (CKKS/BFV), la suma es barata y la multiplicación es cara (aumenta el ruido exponencialmente). En esta transformación, la multiplicación es una operación lineal (suma de logaritmos) y **no expande el ruido de manera multiplicativa**. El error en la suma de logaritmos es aditivo, lo que sugiere un crecimiento de ruido mucho más lento que en CKKS.

La función de reconstrucción P_j^* aplica la inversa (exponencial) y una suma ponderada para recuperar el valor:

$$P_j^*(z) = \sum_{i=1}^{k} w_i(\exp(z_i) - \delta_i^i)$$

Los pesos w_i se optimizan para minimizar el error de aproximación numérica.[1]

3.2.2 Manejo de Circuitos Mixtos y Adición

Dado que la suma no es lineal en el dominio logarítmico ($\log(a + b) \neq \log a + \log b$), el esquema utiliza **transformaciones híbridas** para circuitos aritméticos generales.[1]

$$E_j(x) = (T_L(x), T_M(x), r_j)$$

- T_L : Componente lineal que preserva la estructura aditiva (probablemente una variante aditiva simple o lineal).
- T_M : Componente multiplicativa (la transformación logarítmica descrita anteriormente).

La evaluación homomórfica opera en paralelo sobre estas proyecciones. Cuando se requiere una suma, se opera sobre la proyección T_L ; cuando se requiere un producto, se opera sobre T_M . La función de descifrado P_j^* combina ambos resultados: $P_j^*(z) = F_j(H_j(z_L, z_M))$. Esto implica que el sistema mantiene representaciones redundantes y duales del dato para evadir los compromisos algebraicos de los anillos únicos.

3.2.3 Tratamiento de la No-Linealidad

Para funciones no lineales $f(x)$, en lugar de evaluarlas ciegamente mediante operaciones de anillo (lo que requiere alta profundidad en CKKS), el esquema codifica las potencias desplazadas directamente en el ciphertext [1]:

$$E_j(x) = ((x + \delta_1), (x + \delta_2)^2, \ldots, (x + \delta_d)^d, r_j)$$

Esto permite que la función de evaluación P_j^* reconstruya cualquier aproximación polinomial de grado d como una combinación lineal simple en el momento del descifrado. Esto reduce la complejidad homomórfica de evaluar un polinomio de grado alto de $O(\sqrt{d})$ o $O(\log d)$ multiplicaciones (en esquemas de retículos) a prácticamente $O(1)$ operaciones complejas en el servidor, transfiriendo la carga a la estructura del ciphertext y a la fase de descifrado/evaluación final.

4. Análisis Profundo de Ruido y Estabilidad Numérica

La gestión del ruido es el factor diferenciador más agresivo de la propuesta.

4.1 Modelo de Ruido Determinista y Crecimiento Sublineal

El documento [1] postula un modelo de ruido donde el error acumulado ϵ_L tras L operaciones satisface:

$$\epsilon_L \leq \alpha L^\beta, \quad \text{con } 0 < \beta < 1$$

Este crecimiento **sublineal** contrasta drásticamente con el crecimiento superlineal o exponencial de los esquemas basados en retículos sin *bootstrapping*.

- **En CKKS:** El ruido se multiplica en cada nivel. Si $noise(c) = \epsilon$, entonces $noise(c^2) \approx c \cdot \epsilon$. Tras L niveles, el ruido domina los bits significativos a menos que se

usen módulos gigantescos ($q \approx \Delta^L$).[13]

- **En Transformaciones de Variable:** Al convertir multiplicaciones en sumas (vía logaritmos), el ruido se acumula aditivamente en el dominio logarítmico. Al regresar al dominio lineal mediante la exponencial en P_j^*, la propagación del error depende de la derivada de la función de reconstrucción y de la estabilidad de los pesos w_i.

El teorema 2 del documento sugiere que bajo ciertas condiciones de regularidad (ortogonalidad aproximada de las transformaciones), se puede evitar el *bootstrapping* para una gran clase de circuitos prácticos.[1] Esto eliminaría la operación más costosa de la FHE actual, acelerando potencialmente la inferencia de modelos profundos en varios órdenes de magnitud.

4.2 Optimización de Pesos y Precisión

La precisión del esquema depende de la elección óptima de los pesos w_i en la función de reconstrucción.[1]

$$\epsilon(x, y, \circ) = |P_j^*(E_j(x) \circ E_j(y)) - (x \circ y)|$$

Esta formulación sugiere que el esquema se comporta como un sistema de aprendizaje o interpolación: los parámetros de descifrado (w_i) se "entrenan" o ajustan para minimizar el error en el dominio compacto de entrada D. Esto es fundamentalmente diferente a CKKS, donde el error es un subproducto aleatorio del cifrado LWE que debe ser mitigado ciegamente. Aquí, el error es parte de una función de aproximación determinista que puede ser optimizada.

El documento menciona que el dominio de entrada D es compacto [1], lo que es crucial. Las aproximaciones polinomiales y logarítmicas funcionan bien en intervalos acotados (como en la aproximación de Chebyshev en $[-1, 1]$), pero divergen fuera de ellos. La restricción a un dominio compacto sugiere que el esquema requiere conocer a priori el rango dinámico de los datos, una limitación compartida con CKKS pero gestionada aquí mediante la estructura de la transformación E_j.

5. Seguridad Teórica y Resistencia Cuántica

5.1 Reducción al Problema CVP_α

Mientras que la seguridad de CKKS se basa en RLWE (dureza de encontrar vectores cortos en

retículos ideales ruidosos), el nuevo esquema fundamenta su seguridad IND-CPA en una variante del **Problema del Vector Más Cercano con factor de aproximación** α **(**CVP_α**)**.[1]

El problema consiste en, dado un retículo $\mathcal{L}$ y un vector y, encontrar $v \in \mathcal{L}$ tal que $||y - v|| \leq \alpha \cdot \text{dist}(y, \mathcal{L})$. La implicación es que las transformaciones no lineales $E_j(x)$ producen textos cifrados que, geométricamente, se comportan como puntos perturbados en un espacio de alta dimensión, y recuperar x es equivalente a resolver este problema geométrico.

Si bien CVP es un problema NP-hard en su peor caso y se considera resistente a computadoras cuánticas (al igual que SVP/LWE), la reducción desde un esquema de transformaciones no lineales (logaritmos/potencias) es exótica. La comunidad criptográfica deberá escrutar si la estructura específica de $\Sigma_j(x)$ (logaritmos desplazados) no introduce "atajos" algebraicos que permitan recuperar x sin resolver el problema de retículo general, similar a cómo los ataques algebraicos han roto esquemas multivariados en el pasado.

5.2 El Modelo de Privacidad Funcional

El esquema introduce formalmente la **f-privacidad**, garantizando que un adversario que posee la capacidad de evaluar $P(x)$ (mediante P_j^*) no puede invertir la función para obtener x, siempre que P pertenezca a una clase de funciones autorizadas $\mathcal{F}$.[1] Las condiciones para $\mathcal{F}$ incluyen:

- δ**-no-invertibilidad:** La función debe ser "many-to-one" en un grado suficiente (alta entropía residual $H(X|Y)$).
- **Lipschitz global:** La función no debe ser demasiado sensible a cambios en la entrada, evitando que el adversario aprenda x mediante análisis de gradiente o perturbación.

Esta propiedad es vital para escenarios como el aprendizaje federado, donde el servidor agregador debe poder sumar gradientes ($P(x) = \sum x_i$) pero no debe poder ver los gradientes individuales x_i.[1]

6. Arquitectura del Sistema: Un Pipeline Híbrido

El documento [1] reconoce que la primitiva criptográfica por sí sola no basta y propone un *pipeline* de evaluación segura que integra múltiples tecnologías de mejora de la privacidad

(PETs).

6.1 Componentes del Pipeline

El diseño propuesto es una defensa en profundidad:

1. **AEAD (Authenticated Encryption):** Protege contra la maleabilidad no controlada de los textos cifrados (ataques de *chosen-ciphertext*).
2. **TEE (Trusted Execution Environments):** El uso de enclaves (como Intel SGX) se sugiere para proteger la ejecución de la función de evaluación P_j^* y la gestión de los índices de transformación j. Esto es un reconocimiento pragmático de que la seguridad puramente algorítmica puede necesitar refuerzo de hardware para garantizar la integridad del cómputo.
3. **ORAM (Oblivious RAM):** Se incorpora para ocultar los patrones de acceso a memoria durante la evaluación, cerrando vectores de ataque de canal lateral que son comunes en implementaciones de FHE.
4. **Pruebas STARK:** Se utilizan para verificar la integridad del cómputo ($y' = Eval(c)$) y que la función ejecutada cumple con las políticas de la clase $\mathcal{F}$, sin revelar la función misma (si es propietaria) o los datos.

7. Análisis Comparativo: CKKS vs. Transformaciones de Variable

A continuación se presenta una tabla comparativa sintetizando las diferencias clave halladas en la investigación:

Característica	**Esquema CKKS (Estado del Arte)**	**Nuevo Esquema (Transformaciones de Variable)**
Fundamento Matemático	Aritmética de anillos (RLWE), $\mathbb{Z}_q[X]/(X^N+1)$.	Transformaciones no lineales ($Log, Poly$) y funcionales.
Operación de Multiplicación	Producto polinomial + Reescalado. Expansión de	Suma vectorial en dominio logarítmico. Ruido

	ruido superlineal.	aditivo/sublineal.
Gestión de Ruido	Reescalado constante, Bootstrapping costoso tras L niveles.	Modelo determinista sublineal ($\epsilon \leq \alpha L^{\beta}$). Posible eliminación de bootstrapping.
Funciones No Lineales	Muy costosas. Requieren aprox. polinomial de alto grado.	Nativas o de bajo costo mediante codificación de potencias en el ciphertext.
Seguridad	RLWE (Lattice-based).	CVP_{α} (Lattice-based variant) + f-privacidad.
Decodificación	Recupera $x \approx Dec(c)$.	Funcional: Recupera $P(x) = P_j^*(c)$.
Cuello de Botella	Bootstrapping y Profundidad Multiplicativa.	Suma de textos cifrados (en dominio log) y estabilidad de la inversa P_j^* .

8. Conclusiones y Perspectivas

La investigación revela que el esquema basado en transformaciones de variable aborda directamente las ineficiencias estructurales de los esquemas de retículos actuales. Al transformar la multiplicación en una operación lineal (suma de logaritmos) y al integrar la no linealidad en la estructura del cifrado, promete una eficiencia superior para circuitos profundos y funciones de activación complejas, elementos esenciales para el *Deep Learning* privado.

Ventajas Críticas:

1. **Eliminación del Bootstrapping:** Si el modelo de ruido sublineal se verifica en la práctica,

este esquema podría evaluar redes neuronales profundas sin la pausa masiva de rendimiento que impone el bootstrapping en CKKS.

2. **Flexibilidad Funcional:** La capacidad de adaptar E_j y P_j^* a la función objetivo permite optimizar la precisión para tareas específicas, en lugar de luchar contra la pérdida de precisión genérica de CKKS.

Riesgos y Desafíos:

1. **Complejidad de la Suma:** Mientras que la multiplicación es eficiente, la suma de dos textos cifrados en el dominio logarítmico ($\log(e^a + e^b)$) es no trivial y requiere el uso de transformaciones híbridas, lo que añade complejidad a la orquestación del compilador.
2. **Validación Criptoanalítica:** La seguridad de las transformaciones no lineales es menos comprendida que la de LWE estándar. Existe el riesgo de ataques algebraicos que exploten la estructura de las potencias o logaritmos.

En conclusión, este nuevo paradigma representa una evolución prometedora que podría desbloquear la computación homomórfica para aplicaciones de tiempo real donde CKKS es actualmente demasiado lento o impreciso. Su éxito dependerá de la robustez de su seguridad ante criptoanálisis avanzado y de la estabilidad numérica de sus funciones de reconstrucción en implementaciones de punto flotante de alta precisión.

Nota: Todas las afirmaciones matemáticas y técnicas se basan en el análisis del documento proporcionado [1] y la literatura de referencia sobre el estado del arte.[2]

Obras citadas

1. Paper_FHE_2_0 (2).pdf
2. CKKS Homomorphic Encryption Part 3 - Improved Bootstrapping - MIT, fecha de acceso: enero 13, 2026, https://www.mit.edu/~linust/files/CKKS_Homomorphic_Encryption_Part_3.pdf
3. Fast and Accurate Homomorphic Softmax Evaluation - arXiv, fecha de acceso: enero 13, 2026, https://arxiv.org/html/2410.11184v1
4. On Taylor-like Estimates for Chebyshev Series Approximations of e^x - arXiv, fecha de acceso: enero 13, 2026, https://arxiv.org/html/2401.10169v3
5. CKKS Scheme in Homomorphic Encryption - Emergent Mind, fecha de acceso: enero 13, 2026, https://www.emergentmind.com/topics/ckks-scheme
6. CKKS explained, Part 3: Encryption and Decryption - OpenMined, fecha de acceso: enero 13, 2026, https://openmined.org/blog/ckks-explained-part-3-encryption-and-decryption/
7. Introduction to the CKKS/HEAAN FHE Scheme - KFUPM, fecha de acceso: enero 13, 2026, https://faculty.kfupm.edu.sa/coe/mfelemban/SEC595/References/Introduction%20to%20the%20CKKS-HEAAN%20FHE%20Scheme.pdf

8. Orion: A Fully Homomorphic Encryption Framework for Deep Learning - arXiv, fecha de acceso: enero 13, 2026, https://arxiv.org/html/2311.03470v3
9. CKKS in FHE for operations on approximate numbers - HackMD, fecha de acceso: enero 13, 2026, https://hackmd.io/@rishotics/B1P69977R
10. Part 6: CKKS Scheme - FHE Onchain, fecha de acceso: enero 13, 2026, https://www.encryptonchain.com/blog-post/part-6-ckks-scheme
11. CKKS Encoding and Decoding Explained, fecha de acceso: enero 13, 2026, https://www.imscraping.ninja/crypto/ckks-encoding-and-decoding/
12. CKKS Homomorphic Encryption Part 1 - The Original Scheme - MIT, fecha de acceso: enero 13, 2026, https://www.mit.edu/~linust/files/CKKS_Homomorphic_Encryption_Part_1.pdf
13. CKKS explained, Part 5: Rescaling - OpenMined, fecha de acceso: enero 13, 2026, https://openmined.org/blog/ckks-explained-part-5-rescaling/
14. CKKS Homomorphic Encryption Part 2 - Bootstrapping - MIT, fecha de acceso: enero 13, 2026, https://www.mit.edu/~linust/files/CKKS_Homomorphic_Encryption_Part_2.pdf
15. Homomorphic Multiple Precision Multiplication for CKKS and Reduced Modulus Consumption - ResearchGate, fecha de acceso: enero 13, 2026, https://www.researchgate.net/publication/375809660_Homomorphic_Multiple_Precision_Multiplication_for_CKKS_and_Reduced_Modulus_Consumption
16. Leveraging Discrete CKKS to Bootstrap in High Precision, fecha de acceso: enero 13, 2026, https://ckks.org/blog/2025/high_prec_bootstrap/
17. Homomorphic Evaluation Cluster Architecture for Fully Homomorphic Encryption - IEEE Xplore, fecha de acceso: enero 13, 2026, https://ieeexplore.ieee.org/iel8/8784029/10830546/10993408.pdf
18. (PDF) Efficient Mod Approximation and Its Applications to CKKS Ciphertexts - ResearchGate, fecha de acceso: enero 13, 2026, https://www.researchgate.net/publication/399026648_Efficient_Mod_Approximation_and_Its_Applications_to_CKKS_Ciphertexts
19. Bootstrapping as a Morphism: An Arithmetic Geometry Approach to Asymptotically Faster Homomorphic Encryption - arXiv, fecha de acceso: enero 13, 2026, https://arxiv.org/html/2510.02365
20. SoK: Can Fully Homomorphic Encryption Support General AI Computation? A Functional and Cost Analysis - arXiv, fecha de acceso: enero 13, 2026, https://arxiv.org/html/2504.11604v2
21. Fast and Accurate Homomorphic Softmax Evaluation | Request PDF - ResearchGate, fecha de acceso: enero 13, 2026, https://www.researchgate.net/publication/384938734_Fast_and_Accurate_Homomorphic_Softmax_Evaluation
22. Efficient Homomorphic Evaluation on Large Intervals - ResearchGate, fecha de acceso: enero 13, 2026, https://www.researchgate.net/publication/361754368_Efficient_Homomorphic_Evaluation_on_Large_Intervals
23. Power-Softmax: Towards Secure LLM Inference over Encrypted Data - alphaXiv, fecha de acceso: enero 13, 2026, https://www.alphaxiv.org/overview/2410.09457v1

24. On the Precision Loss in Approximate Homomorphic Encryption - ResearchGate, fecha de acceso: enero 13, 2026, https://www.researchgate.net/publication/378301079_On_the_Precision_Loss_in_Approximate_Homomorphic_Encryption
25. FHE: What is the difference between multiplicative depth and multiplicative level? - Cryptography Stack Exchange, fecha de acceso: enero 13, 2026, https://crypto.stackexchange.com/questions/72982/fhe-what-is-the-difference-between-multiplicative-depth-and-multiplicative-leve